

Übungsblatt 6: Multicore-Echtzeit und Shared Memory

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Zielsetzung

In dieser Übung konfigurieren Sie ein asymmetrisches Multicore-System (**AMP**) auf dem **STM32H755**. Der Cortex-M7 fungiert als Bare-Metal-Master für hohen Durchsatz, während der Cortex-M4 als Echtzeit-Kern **FreeRTOS** ausführt. Sie etablieren einen Ping-Pong-Datenaustausch über einen Shared-Memory-Bereich im **SRAM4** und provozieren sowie beheben den klassischen *Cache Coherency Failure* mithilfe der **Memory Protection Unit (MPU)** auf Hardware-Ebene.

Aufgabe 1: Dual-Core-Setup und Boot-Prozess

Im Gegensatz zu klassischen SMP-Systemen starten beim STM32H755 nicht beide Kerne gleichzeitig. Der Cortex-M7 bootet als Master, während der Cortex-M4 im Reset-Zustand verharrt.

1. Starten Sie ein neues Projekt für das Board **STM32H755ZI-Q** in der STM32CubeIDE. Aktivieren Sie beide Prozessorkerne.
2. Aktivieren Sie FreeRTOS ausschließlich für den Cortex-M4 (*Asymmetric OS Design*).
3. Ergänzen Sie den generierten C-Code so, dass der Cortex-M7 den Cortex-M4 beim Systemstart aufweckt.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 2: Shared Memory und Linker-Script

Die Kerne benötigen einen gemeinsamen Speicherbereich. Nutzen Sie den **SRAM4** in der D3-Domäne ab der Adresse `0x38000000` (64 KB), da dieser für beide Kerne gut erreichbar ist.

1. Legen Sie eine Header-Datei `shared_data.h` im Ordner `Common/Inc` an, die ein C-Struct mit den Flags `m7_ready`, `m4_ready` und einem `payload_value` enthält.
2. Deklarieren Sie die Shared-Variable mit dem Schlüsselwort `volatile` und weisen Sie sie per GCC-Attribut der Sektion `.shared_memory` zu. Legen Sie die Definition (ohne `extern`) in genau einer `.c`-Datei im Common-Ordner an und binden Sie den Header in beiden Projekten ein.
3. Öffnen Sie die Linker-Skripte (`.ld`) **beider** Kerne und fügen Sie den Speicherbereich (RAM_D3 ab `0x38000000`, 64 K) sowie die Sektion `.shared_memory` hinzu. Achten Sie zwingend auf das Flag (NOLOAD)!

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 3: Ping-Pong-Logik (Inter-Core-Kommunikation)

Implementieren Sie ein asymmetrisches Handshake-Protokoll zwischen den Kernen.

1. **Cortex-M7 (Bare-Metal):** Initialisieren Sie die Werte (`payload_value = 0`, `m7_ready = 1`). Warten Sie in der Super-Loop per Polling darauf, dass der M4 geantwortet hat (`m4_ready == 1`). Geben Sie den Wert per `printf()` aus, quittieren Sie das Flag, erhöhen Sie den Wert, warten Sie 500 ms zur Lesbarkeit und übergeben Sie wieder an den M4 (`m7_ready = 1`).
2. **Cortex-M4 (FreeRTOS):** Die RTOS-Task soll auf Daten des M7 warten, solange `m7_ready == 0`. Nutzen Sie dabei `osDelay(1)`, damit die Task die CPU nicht per Busy-Wait blockiert. Sobald Daten anliegen, quittieren Sie das M7-Flag, inkrementieren Sie den Payload und signalisieren Sie die Antwort (`m4_ready = 1`).

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Aufgabe 4: Cache-Kohärenz und System-Freeze

Wenn Sie den Code aus Aufgabe 3 flashen, friert das System nach der initialen Ausgabe ein: Die Konsole bleibt stumm, der Cortex-M4 verharrt in seiner Warteschleife.

1. Erklären Sie physikalisch, warum dieser *Cache Coherency Failure* auftritt.
2. Lösen Sie das Problem hardwareseitig über die Memory Protection Unit (MPU) des Cortex-M7 im CubeMX-Configurator. Geben Sie die genauen Konfigurationsparameter an.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)